Learning Competitive Decision-Making in Pokemon Battles Using Deep Neural Networks

Nick Bolger

1 Introduction

Competitive Pokemon battles are a useful testbed for studying sequential decision-making under uncertainty. On each turn, a player must choose among several actions while reasoning about hidden information, type matchups, move effects, and longer-term game outcomes. This kind of structured decision-making problem is relevant beyond games because many real-world systems also require selecting actions based on incomplete state information and delayed rewards.

Existing Pokemon battle agents are often based on hand-crafted heuristics, simple damage calculations, or hard-coded strategies. While these approaches can perform reasonably well in narrow settings, they may struggle to generalize across diverse battle states or learn more subtle policies that consider long-term tradeoffs. This motivates exploring whether a neural network-based agent can learn stronger action-selection behavior directly from battle state representations.

In this project, I propose to build a Pokemon Showdown battle agent that uses deep learning to select actions in a competitive battle environment. The main idea is to represent the current battle state as a feature vector and train a neural model to choose among legal actions, such as selecting one of four moves or switching to another Pokemon. The project will begin with a simple baseline model and compare it against random and heuristic-based agents. The core hypothesis is that a neural decision model trained through repeated interaction with the battle simulator can outperform non-learning baselines in terms of win rate and decision quality.

2 Related Work

A first cluster of related work comes from research on reinforcement learning for games and sequential decision-making. Prior work in this area shows that learned policies can outperform manually specified strategies when the environment has a clear state, action space, and reward signal. These methods are especially relevant because Pokemon battles involve turn-by-turn action selection, delayed rewards, and the need to balance short-term and long-term outcomes.

A second cluster of related work consists of existing battle engines, Pokemon simulators, and bot implementations built on Pokemon Showdown or similar platforms. Many existing bots rely on rule-based logic, damage estimates, and manually designed priorities. These systems demonstrate that the environment is suitable for automation, but they often do not use learned state representations or deep models for policy selection.

A third cluster includes broader work on neural policies in turn-based games. Compared with those works, this project focuses on a domain with a mix of visible and partially hidden information, a constrained action space, and rich strategic interactions. My work differs from prior heuristic Showdown bots because it aims to learn decision policies from data or simulated experience instead of depending entirely on manually specified battle rules.

A fourth consideration specific to the Pokemon domain is the challenge of teambuilding. In standard competitive formats, performance depends not only on in-battle decision-making but also on constructing an effective team beforehand. This introduces an additional layer of complexity that may confound evaluation of a battle agent, since poor team composition can overshadow good decision policies. To mitigate this issue, this project will likely begin with the "Random Battles" format, which is already highly popular on Pokemon Showdown, in which teams are automatically generated, allowing the focus to remain on action selection during battle. At the same time, teambuilding itself represents a promising extension of this work: a similar neural approach could be applied to learning team composition strategies in formats where players select their own teams, framing team construction as a separate optimization or policy-learning problem.

3 Methods

The implementation will use Pokemon Showdown as the battle environment. I plan to either connect to an existing Python interface for Pokemon Showdown or build a lightweight interface that can read battle state information and submit legal actions. To keep the scope manageable, the initial version of the project will likely focus on a restricted format, such as one battle tier or a simplified team setting, rather than attempting to support every battle format.

The first technical task is to define a state representation for the model. The representation will include information such as the active Pokemon on each side, current HP values, type information, status conditions, stat boosts, available moves, and the set of legal actions on the current turn. If possible, additional features such as revealed opponent team members and field conditions will also be included. These features will be encoded into a fixed-length numeric input vector.

The initial model will be a multilayer perceptron that takes the encoded battle state as input and outputs scores or probabilities over legal actions. This simple architecture is appropriate for a first version because it is easy to implement and provides a clear baseline for later experiments. Depending on progress, I may extend the project to compare multiple architectures or train the model using reinforcement learning methods such as Deep Q-Learning or policy-gradient-based updates.

Training will be done through repeated simulated battles. The agent may learn through self-play, through play against a random baseline, or through play against a heuristic baseline. A reader should be able to reproduce the setup by using the same environment, battle format, state encoding rules, neural network architecture, and training procedure described in the final report.

4 Experiments

The first planned experiment is a baseline comparison. I will evaluate three agents: a random-action agent, a heuristic agent, and the neural-network-based agent. The main purpose of this

experiment is to measure whether the learned model provides a meaningful improvement over non-learning approaches. The primary evaluation metric will be win rate over a fixed number of simulated battles. Secondary metrics may include average battle length, action-selection latency, and the proportion of games lost due to clearly poor tactical choices.

The second planned experiment is an architecture or training comparison. For example, I may compare a small multilayer perceptron against a larger one, or compare supervised imitation-style training against reinforcement learning updates if both are feasible within the project timeline. The purpose of this experiment is to determine whether additional model complexity or a different learning strategy produces better battle performance. Evaluation will again focus primarily on win rate, with training time and computational cost used as supporting metrics.

These experiments are designed to validate the project's main contribution: demonstrating that a neural approach can learn better decision policies for Pokemon battles than simple baselines. Because the setting is simulated and repeatable, the evaluation should be objective and reproducible.

Bibliography

The bibliography will include at least four sources: work on reinforcement learning in games, work on neural decision-making or game-playing agents, documentation or prior tools related to Pokemon Showdown bots, and any papers used to motivate the chosen model or evaluation method.